Autocallable Options Pricing with Integration-Based Exponential Amplitude Loading

Francesca Cibrario*, Ron Cohen†, Emanuele Dri‡, Christian Mattia*, Or Samimi Golan†, Tamuz Danzig†, Giacomo Ranieri*, Hanan Rosemarin†, Davide Corbelletto*, Amir Naveh†, Bartolomeo Montrucchio§

*Data, AI and Technology, Intesa Sanpaolo, Torino, Italy
francesca.cibrario@intesasanpaolo.com, christian.mattia@intesasanpaolo.com,
giacomo.ranieri@intesasanpaolo.com, davide.corbelletto@intesasanpaolo.com
†Classiq Technologies, Tel Aviv, Israel
ron@classiq.io, orsa@classiq.io, tamuz@classiq.io, hanan@classiq.io, amir@classiq.io
‡Fondazione LINKS, Torino, Italy
emanuele.dri@linksfoundation.com
§DAUIN, Politecnico di Torino, Torino, Italy
bartolomeo.montrucchio@polito.it

arXiv:2507.19039v1 [quant-ph] 25 Jul 2025

***Abstract*—We present a comprehensive quantum algorithm tailored for pricing autocallable options, offering a full implementation and experimental validation. Our experiments include simulations conducted on high performance computing (HPC) hardware, along with an empirical analysis of convergence to the classically estimated value. Our key innovation is an improved integration-based exponential amplitude loading technique that reduces circuit depth compared to state-of-the-art approaches. A detailed complexity analysis in a relevant setting shows a ∼50x reduction in T-depth for the payoff component relative to previous methods. These contributions represent a step toward more efficient quantum approaches to pricing complex financial derivatives.**

***Index Terms*—derivative pricing, autocallable option, quantum computing, amplitude loading, quantum finance**

I. Introduction

Quantum computing is widely expected to benefit several specific types of computationally intensive problems. Derivative pricing, along with other tasks in the financial sector, is an excellent candidate to test the potential and limitations of the quantum computing paradigm [1]–[4]. Various studies and contributions focus on the implementation and analysis of quantum algorithms to price different types of structured products.

Several existing proposals hint at the possibility of achieving a quantum advantage over the traditional Monte Carlo approach in the asymptotic regime for *path-independent* financial products, such as European options. Although these algorithms constitute a fundamental first step toward the study and analysis of more complex derivatives, these products do not necessarily present a significant challenge for traditional pricing methodologies. There is room to explore genuine practical utility in the design of quantum algorithms to price more complex classes of products, such as *path-dependent* derivatives. Autocallable options, which are path-dependent financial products, are ideal for quantum experiments. They have already been used for benchmarking in [5], [6] to assess the potential and requirements to achieve quantum advantage in this area.

Specifically, Chakrabarti et al. [5] proposed a novel reparameterization method that integrates pre-trained variational circuits with fault-tolerant quantum computing to lower the amount of resources needed for achieving quantum advantage. More recently, work by Stamatopoulus and Zeng [6] focused on limiting the quantum arithmetic components needed for derivative pricing on quantum computers. To avoid quantum arithmetic, their methodology leverages Quantum Signal Processing (QSP) by encoding the derivative's payoff directly into quantum amplitudes. This approach, effectively diminishes the requirements for achieving quantum advantage. Cibrario et al. [7] proposed an algorithm to price rainbow options that reduces the use of quantum arithmetic through an alternative approach to QSP. This decreases the depth of the module that computes the payoff as an amplitude but affects the normalization factor required for the entire algorithm. We propose an enhancement to their exponential amplitude loading methodology to build a novel end-to-end algorithm for pricing autocallable options. The algorithm offers improvements in the depth of the corresponding quantum circuit relative to the state-of-the-art. In addition, we built quantum circuits to test our algorithm using the Qmod language [8]. We simulated the circuits for a small instance of the autocallable pricing problem to validate the results against a classical benchmark. We conducted experiments with up to 33 qubits using the LEONARDO pre-exascale supercomputer [9]. The proposed methodology and results represent a step towards the efficient pricing of complex derivative products, which is essential to achieve *quantum utility*, a practical advantage of quantum computing over classical methods. This challenge remains one of the most significant in the field today.

The remainder of this paper is structured as follows. Section II provides a basic overview of derivative pricing (with a focus on autocallable options) and the related quantum

methods existing in the literature. Section III describes the novel implementation that constitutes the main contribution of this work, while Section IV analyzes the complexity of the building blocks of our method. Then, Section V presents the results of an extensive simulation campaign aimed at validating the proposed approach and illustrating the effects of scaling. Finally, Section VI summarizes the main outcomes of this work and illustrates potential future avenues for further development.

II. Quantum Derivative Pricing

A derivative is a financial contract in which two parties agree on a future transaction. This kind of contract depends on the value of its underlying variables, such as stocks, commodities, currencies, or market indexes. Some derivatives, such as forwards and futures, impose the obligation to buy or sell the underlying variables. Options grant the right to buy or sell, but not the obligation [10]. Determining the price of options is a crucial task for financial institutions.

An option can be a *call* or a *put*, depending on whether it grants the right to buy or sell the underlying variables by a certain date (i.e., *maturity date*) and at a given price (i.e., *strike price*). European options allow the contract owner to exercise this right only upon the maturity date, while American options can also be bought or sold before that specific point in time. An option can be defined based on a single underlying variable (*single-asset*) or on multiple variables (*multi-asset*). The payoff is the profit obtained by the option holder when the contract ends. If exercising the option is not advantageous, the payoff is zero and the option is considered *out of the money*; otherwise, the payoff is positive and the option is said to be *in the money*. The payoff can depend solely on the value of the asset's price on the maturity date (*path-independent option*) or on the path the asset price follows over the entire life of the contract (*path-dependent option*).

Pricing an option means assigning a fair value to the contract by estimating the expected discounted payoff. Discounting a future payoff means evaluating its present value. Given a payoff f at time t, its value today is given by $f \cdot e^{-rt}$, where r is the risk-free interest rate.

Multi-asset path-dependent options are challenging to price, since the calculation must be performed by modeling the distribution of correlated asset prices over multiple time steps.

Different models can be adopted to represent these distributions. One such example is the Black-Scholes model, which assumes the following dynamics for the evolution of asset price:

$$dS_t = \hat{\mu} S_t dt + \hat{\sigma} S_t dW_t \tag{1}$$

where S_t is the asset price at time t, dW_t is a Wiener process, $\hat{\mu}$ is the drift, and $\hat{\sigma}$ is the volatility [10]. In this type of model, $\hat{\sigma}$ is constant, while in other more realistic models, it is considered stochastic or local. The parameters of the Black-Scholes model can be estimated easily by observing historical data, while local and stochastic volatility models require calibration. Only under certain circumstances is it possible to price an option in a closed form by solving the stochastic differential equation or determining an analytical solution.

Monte Carlo simulation can be used as an alternative method to estimate the expected discounted payoff, especially when a closed form does not exist. It constructs different paths (scenarios) for asset price evolution over discretized time steps. Specifically, the asset price evolution of Equation 1 can be discretized as:

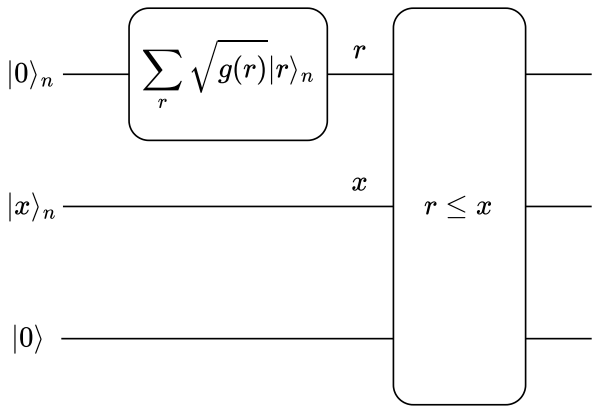
$$S(t + \Delta t) = S(t) e^{\mu \Delta t + \sigma \alpha \sqrt{\Delta t}} \tag{2}$$

where $\mu = \hat{\mu} - \frac{\hat{\sigma}^2}{2}$, $\sigma = \hat{\sigma}$, and α is a random extraction from a standard normal distribution (asset prices are log-normally distributed). Different scenarios can be built by sampling different random paths. Once the value of the asset price is known for all time steps, the discounted payoff can be calculated for each scenario and then averaged across all paths to estimate the expected payoff. The convergence rate of this classical method is $1/\sqrt{M}$, where M is the number of scenarios simulated.

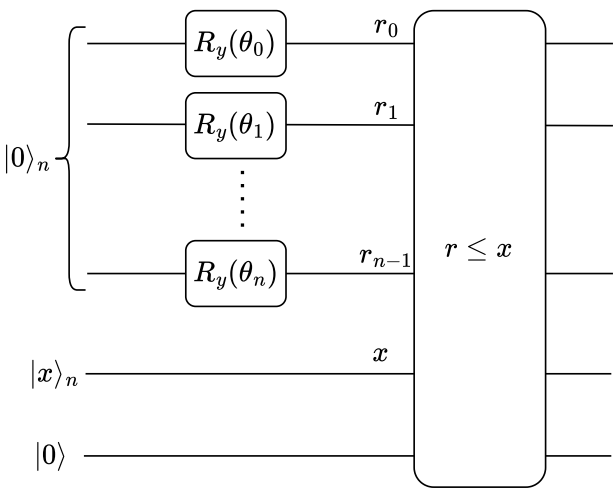
Quantum computing promises a quadratic speed-up compared to the Monte Carlo approach by leveraging the Quantum Amplitude Estimation (QAE) algorithm [11]. The quantum approach provides a convergence rate of $1/N$, where N is the number of calls to the Grover operator, which is computationally analogous to a Monte Carlo path evaluation. Building on this theoretical result, several studies have focused on the pricing problem, with the Black-Scholes model serving as a primary framework. Rebentrost, Gupt and Bromley [12] presented a quantum approach for pricing single-asset derivatives. Other subsequent works have addressed the challenge of extending the range of options that can be priced using quantum computing, while reducing the complexity of each sub-module in the quantum algorithm to fully exploit the quadratic speed-up. Stamatopoulos et al. [13] analyzed a wide range of derivatives, including multi-asset and path-dependent ones, loading the log-normal distribution of asset prices into a quantum state. Chakrabarti et al. [5] provided an estimate of the resources required to achieve quantum advantage in derivative pricing. They also proposed an optimization of the probability distribution encoding module by loading the log-returns distribution instead of the asset prices one. This approach introduced the need to transition from the log-return space to the price space using quantum arithmetic. In [6] and [7], two strategies were proposed to delay the transition and avoid the need for quantum arithmetic. Other studies have focused on extending the quantum pricing approach beyond the Black-Scholes model, considering models with local volatility [14] or stochastic volatility [15], without addressing the calibration phase.

A. Autocallable Options

An *automatically callable* (or autocallable) option is a structured financial derivative instrument designed to provide potential payouts based on specific conditions met during its lifespan. Autocallable options are popular in structured finance



(a) Circuit implementing the integration amplitude loading. The whole circuit is composed of two sub-modules: (1) a state preparation loading a g function into the amplitude and (2) a comparator. The comparator ($r \leq x$) behaves like an integrator of the g function with x as its limit. The result of the integral is collected in the amplitude of the $|1\rangle$ state of the target qubit (i.e., the qubit storing the outcome of the comparison).



(b) Circuit implementing the integration amplitude loading for the exponential function reported in Equation 13. It consists in implementing the exponential state preparation with n parallel R_y with rotation angles defined in Equation 12.

Fig. 1: Integration amplitude loading circuit for a generic function (a) and for the exponential one (b).

and computationally expensive to price: this provides strong motivation to explore the potential of quantum computing in pricing such options. Autocallable options are characterized by the following features:

- **Single or Multi-asset:** Their performance can be tied to a single underlying asset or a basket of assets. With multiple assets, one of them can be used as the reference for payoff calculation; generally, this will be either the best-performing (*best-of*) or worst-performing (*worst-of*) asset.
- **Path-dependency:** The asset has predefined dates when its performance is evaluated, creating multi-step observation periods (e.g., monthly or quarterly) that represent opportunities to trigger a payout.
- **Notional Value:** Autocallables are typically defined in terms of asset returns rather than prices, and, therefore, in terms of strike returns instead of strike prices. Payouts are calculated as a fraction of a notional amount, V, which determines the contract size in monetary units (it may be equal to the asset's initial price).
- **Binary Payoffs:** The derivative includes a sequence of binary options. A binary option's payoff is a fixed amount if a condition is met; otherwise, the payoff is 0. These options, in the context of autocallable derivatives, are linked to specific dates within the multi-step observation period. Each i-th binary option has an expiration date t_i upon which the condition is checked and the payoff is computed. If the asset return r_{t_i} at t_i, exceeds a strike return k_i, the binary option is considered in the money and pays out a fixed amount f_i. This payout can also be expressed as a percentage of the notional value. When a binary option triggers a payout, the contract is immediately terminated.
- **Short Knock-in Put:** Along with the binary options, the derivative contract includes a short knock-in put option that expires on maturity date T. Since the autocallable contract terminates if any binary option is in the money, the put option is only evaluated if none of the binary options have provided a payout. The payoff of the put option depends on the difference between the asset return and the strike return K. Due to its knock-in nature, one condition for a non-zero payoff is that the asset return r_t crosses a barrier b at least once during the option's lifetime. The short position means that the option holder must pay the issuer if the payoff is non-zero. The payoff related to the put option is given by:

$$f = \begin{cases} V\left(r_T - K\right), & \text{if } r_T < K \text{ and } r_t < b, \forall t \in [0, T] \\ 0, & \text{otherwise} \end{cases}$$

where r_T the return at the final time T.

B. Quantum Approach

The quantum approaches for derivative pricing usually focus on two steps: probability distribution encoding and payoff evaluation. First, a probability distribution over discretized asset paths is loaded into a quantum register through a state preparation operator. Let ω denote a discretized path. This paths' state preparation operator creates the following state:

$$\sum_{\omega} \sqrt{p(\omega)}\, |\omega\rangle, \tag{3}$$

where $p(\omega)$ is the probability of path ω. Second, the discounted payoff function rescaled between 0 and 1 ($\tilde{f}$) is calculated and encoded in the amplitude of a target qubit.

$$\sum_{\omega} \sqrt{\tilde{f}(\omega)\, p(\omega)}\, |\omega\rangle\, |1\rangle + \sum_{\omega} \sqrt{(1-\tilde{f}(\omega))p(\omega)}\, |\omega\rangle\, |0\rangle \tag{4}$$

The probability of measuring the target qubit in the state $|1\rangle$ is:

$$E[\tilde{f}] = \sum_{\omega} p(\omega)\tilde{f}(\omega) \tag{5}$$

which is the rescaled expected discounted payoff of the option.

Through Quantum Amplitude Estimation (QAE), an estimation of $\mathbb{E}[\tilde{f}]$ can be performed with a query complexity of $O(1/\epsilon)$, where ϵ is the desired error tolerance. This step ultimately provides a quadratic speedup compared to the classical Monte Carlo complexity of $O(1/\epsilon^2)$. The QAE technique requires the use of an additional quantum register to encode the estimate. However, certain extensions of the methodology have been developed to circumvent this requirement. A notable example is the Iterative Quantum Amplitude Estimation (IQAE) as discussed in [16].

Regarding the paths' state preparation, the *re-parameterization method* proposed in [5] exploits the fact that log-returns follow a normal distribution. They used variationally pre-trained Gaussian loaders (G) to load the standard normal distribution:

$$G : |0\rangle \longrightarrow \sum_{g=0}^{2^n-1} \sqrt{p(g)}\, |g\rangle \tag{6}$$

where $p(g)$ are the discretized probabilities of a standard normal distribution (with 2^n discretization points). Affine transformations are then used to obtain the desired mean and standard deviation, exploiting the following reformulation of Equation 2:

$$\hat{l}(t) = \ln \frac{S(t+\Delta t)}{S(t)} = \mu\Delta t + \sigma\alpha\sqrt{\Delta t} \tag{7}$$

Using this approach, they had to apply quantum arithmetic to compute log-returns by accumulation $l(t) = \sum_{x=0}^{t} \hat{l}(x)$ and then transition from the log-return space to return space $r(t) = e^{l(t)}$.

To avoid the use of expensive quantum arithmetic, techniques such as Quantum Signal Processing (QSP) have been applied to create a unified payoff component that handles both the transition from log-return space to return space and the direct encoding of the payoff function into quantum amplitudes [6]. To compute a non-constant payoff, the state is first prepared using a unitary operator U_{sqrt} that encodes a scalar number u into an amplitude:

$$U_{sqrt}\, |u\rangle|0\rangle \longrightarrow |u\rangle \left(\sqrt{u}\, |\psi_0\rangle|0\rangle + \sqrt{1-u}\, |\psi_1\rangle|1\rangle\right). \tag{8}$$

QSP is then used to apply a polynomial transformation to encode an approximation of the function in the amplitude of the QAE target state:

$$\tilde{f}(u) = \sqrt{\frac{Ae^u - B}{C}}, \tag{9}$$

where the constants A, B, and C are used to compute the payoff and to ensure $\tilde{f}(u) \in [0, 1]$.

In the context of autocallable options, the authors claimed to reduce the logical clock rate needed for quantum advantage by a factor of ~ 5x with respect to [5]. Moreover, they claim a reduction of the required T-gates by $\sim$ 16x and of the necessary logical qubits by ~ 4x.

In this work, we propose an alternative method that requires fewer resources than the QSP-based approach. We enhance the integration-based exponential amplitude loading methodology introduced in [7] (detailed in Section III-A) to avoid degrading the rescaling factor needed in the post-processing operations.

III. Methodology

We propose a quantum algorithm for pricing autocallable options (path-dependent derivatives) and provide a concrete implementation for the pricing of a single asset while generalizing the complexity analysis for the multi-asset case. The developed algorithm leverages the exponential amplitude loading technique introduced in [7] to price multi-asset path-independent rainbow options. The authors introduced two methods to implement exponential amplitude loading: direct and integration. We propose an improvement of the integration amplitude loading to enhance the postprocessing mapping of the expected value. This serves to reduce the overall accuracy requirements for the quantum algorithm compared to [7]. Section III-A and III-B discuss the integration amplitude loading technique independently of its application to option pricing. The symbols used herein are defined locally and do not correspond to those in other sections.

A. Integration Amplitude Loading

Given a quantum state $|x\rangle$ composed of n qubits and interpreted as an integer, the integration amplitude loading technique introduced in [7] enables the encoding of a summation over a pre-loaded probability distribution into the amplitude of the state. As depicted in Figure 1a, it uses n additional qubits prepared with the state $\sum_r \sqrt{g(r)}|r\rangle$ plus one auxiliary qubit and a comparator to create the following state:

$$\sqrt{\sum_{r=0}^{x} |g(r)||x\rangle_n|\psi\rangle_n|1\rangle} + \sqrt{\sum_{r=x+1}^{2^n-1} |g(r)||x\rangle_n|\psi_\perp\rangle_n|0\rangle} \tag{10}$$

where $|\psi\rangle_n$ and $|\psi_\perp\rangle_n$ are two orthogonal states. The comparator basically performs the integral of the function $g(r)$. To load an exponential function, $g(r)$ is defined as:

$$g(r) = \frac{e^{ar}}{Z} \tag{11}$$

where Z is the normalization factor ensuring the state in the r register is properly normalized. In this specific context, Z is equal to $\sum_i e^{ai}$. Loading this state requires only a set of n parallel R_y rotations, each with angle:

$$\theta_i = 2\arctan\left(e^{\frac{a2^i}{2}}\right), \tag{12}$$

as shown in Figure 1b. With this exponential state preparation, the amplitude of the $|1\rangle$ state of the auxiliary qubit in 10 becomes:

$$\sqrt{\frac{e^{a(x+1)}-1}{e^{a2^n}-1}} \tag{13}$$

If the input x is limited to only a subset of the possible states of its quantum register, the normalization factor $e^{a2^n}-1$ becomes excessive, as it is designed for the full range of 2^n states. Since only a restricted input domain is considered, this overestimation negatively affects the accuracy of subsequent estimates.

B. Improving Integration Amplitude Loading

We suggest a mitigation for the normalization issue of the method described in III-A by loading partial exponential states. By preparing the register r with an exponential state, but only for the relevant range (assuming it is continuous), the normalization is optimal. Denoting the limits of integration by $x_0 < x_1 \in [0, 2^n-1]$, loading the register r with the state:

$$\sum_{x_0}^{x_1} \sqrt{\frac{e^{ar}}{Z}}|r\rangle \tag{14}$$

results in the following amplitude after integration:

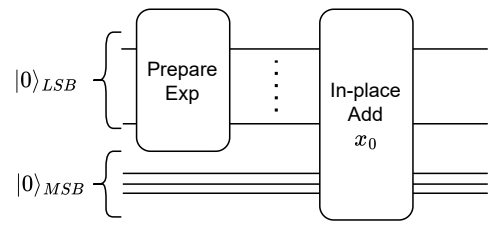
$$\begin{cases} 0, & \text{if } x < x_0, \\ \sqrt{\frac{e^{a(x+1)}-e^{ax_0}}{e^{a(x_1+1)}-e^{ax_0}}}, & \text{if } x_0 \le x \le x_1, \\ 1, & \text{if } x > x_1. \end{cases} \tag{15}$$

.

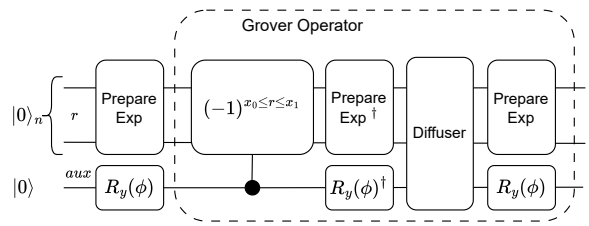
We now describe the procedure for preparing a partial exponential state, given by $\sum_{x_0}^{x_1} \sqrt{\frac{e^{ar}}{Z}}|r\rangle$ for a specified x_0 and x_1. Exponential state preparation for the entire domain of a quantum variable, as described in [7], can be done using a single R_y rotation on each qubit, as depicted in Figure 1b. For a limited domain $[x_0, x_1]$, if the difference $x_1 - x_0$ is a power of 2, it can be efficiently loaded by exponential state preparation on the LSB[1] of $|r\rangle$ followed by an in-place addition if needed, as shown in Figure 2a. If the difference is not a power of 2, then the partial exponential state preparation includes a complete exponential state preparation on all qubits, followed by a single round of exact amplitude amplification as described in [17], [18]. The Grover operator should use a suitable comparator as an oracle, as shown in Figure 2b.

In the context of option pricing, the exponential function is needed to shift from the log-return space to the return space, as the payoff is commonly expressed in terms of returns or asset prices. We highlight that the entire state preparation on the r register can be parallelized with other parts of the quantum circuit, leaving only the comparator to be done serially.

[1]Least Significant Bits (whereas MSB stands for Most Significant Bits).



(a) Implementation for interval size which is a power of 2. The addition should be used in the case $x_0 \neq 0$.



(b) Implementation when interval size is not a power of 2. The circuit uses the exact Amplitude Amplification scheme described in [18]. The angle ϕ is chosen such that the amplification is exact. The auxiliary is freed by the amplification. If the interval $[x_0, x_1]$ does not contain most of the probability, it is possible to replace the block 'Prepare Exp' with the module in 2a to use a single amplification round.

Fig. 2: State preparation of exponential on a specific interval: $\sum_{x_0}^{x_1} \sqrt{\frac{e^{ar}}{Z}}|r\rangle$.

C. Autocallable Option Pricing

We now detail a complete description of our method for pricing a single-asset autocallable option. With the addition of some arithmetic modules, the approach is fully extensible to the case of the *best-of* or *worst-of* basket autocallable. We applied Algorithm 1 to obtain a quantum circuit generating a state in the form of Equation 4 and used IQAE [16] to estimate the probability of the target qubit being in the $|1\rangle$ state. Based on the *re-parameterization method* described in [5] we loaded standard Gaussians (one for each time step) and then used arithmetic to adjust the mean and variance. Log-returns for each time step could then be calculated by accumulation using an in-place adder. We applied all the necessary comparators in the log-return space to check the binary options and the put option conditions. Instead of transitioning to the return space using arithmetic, we used the integration amplitude loading described in Section III-B.

The estimation target for the whole algorithm is:

$$\bar{f} = E[f_1 e^{-rt_1} i_1 + ... + f_j e^{-rt_j} i_j + f e^{-rT} i_{j+1} + 0 i_{j+2}]$$

where f_k is the k-th binary's payoff and f is the put option non-zero payoff. The rescaling e^{-rt} is necessary to discount

the payoff, and it can be included directly in the loaded payoff. i_k are indicator functions defined for mutually exclusive events. The first is set to 1 just in case the first binary is in the money. From the second to the j-th the indicators are set to 1 only if the indexed binary is in the money and if all of the previous ones were out of the money. i_{j+1} is set to 1 if the put option is activated in the money, and i_{j+2} is set to 1 if the put option is activated out of the money.

Because the amplitude estimation returns a number in the range $[0, 1]$, we needed to rescale the payoff values to this range. In order to take full advantage of the range, we mapped an amplitude of 0 to the minimum payoff value, $(r_{T_{\min}} - K)V$, and an amplitude of 1 to the maximum payoff, $f_{\max} = \max_i\{f_i e^{-rt_i}\}$. This mapping results in a normalization factor of $f_{\max} + (K - r_{T_{\min}})V$, where $r_{T_{\min}}$ is the minimum possible value of the return at time T. Note that in this mapping, a zero payoff requires loading a number greater than 0 into the amplitude.

Loading the constant payoffs ($f_i e^{-rt_i}$ and the zero payoff) according to the mapping was achieved using a single R_y on an indicator qubit.

The non-zero payoff loading for the put option was done with the integration amplitude loading mentioned in III-B. The reference variable was loaded with an exponential distribution in the range of the put option non-zero payoff, between the minimum log-return and the strike log-return $(\ln K)$. Finally, we performed an R_y rotation on an additional indicator qubit to map the normalization factor from $(K - r_{T_{\min}})V$ to $f_{\max} + (K - r_{T_{\min}})V$. An alternative to re-mapping the put option non-zero payoff is to load the exponential reference state including the entire payoff range (considering binary payoffs). This approach avoids the use of the additional indicator qubit.

IV. Complexity Analysis

Because fault-tolerant implementations of the T gate are resource intensive, it is crucial to minimize the T-depth in quantum algorithm development. In this section, we analyze the T-depth complexity of the whole algorithm, while considering the required accuracy [19]. We reduce the total T-depth compared to other proposed implementations of the same application [5], [6].

All sources of error in the algorithm should be bounded by the desired probability error threshold, ϵ:

$$\epsilon_{\text{truncation}}, \epsilon_{\text{discretization}}, \epsilon_{\text{approximation}}, \epsilon_{\text{arithmetic}}, \epsilon_{\text{amplitude loading}} \leq \epsilon$$

where $\epsilon_{\text{truncation}}$, $\epsilon_{\text{discretization}}$ and $\epsilon_{\text{approximation}}$ are the Gaussian preparation's truncation, discretization, and approximation errors, respectively, while $\epsilon_{\text{arithmetic}}$ and $\epsilon_{\text{amplitude loading}}$ are the errors resulting from the arithmetic and amplitude loading modules respectively. For a given error threshold, one must choose the appropriate errors that do not exceed the threshold.

We analyze the whole algorithm's complexity by decomposing it into consequential building blocks, summing each one's T-depth, and considering the required error and the rescaling.

The whole T-depth is as follows:

$$D_{\text{tot}} = (1 + 2N_{\text{IQAE}}) \cdot (\max(D_{\text{G}} + D_{\text{arith}}, D_{\text{exp}}) + D_{\text{AL}})$$

where:

- N_{IQAE} is the total number of calls to the operator building the state in Equation 4. It is multiplied by 2 because it appears twice in the Grover operator. The value of N_{IQAE} is chosen according to ϵ by $N \sim 1/\epsilon$.
- D_{G} is the T-depth of a Gaussian preparation block. All Gaussian preparation blocks, one for each timestep and asset, are called in parallel. The overall error is therefore multiplied by T, the number of time steps and d, the number of assets. To ensure the error bounding of the parallel modules, D_{G} requests an error of $\epsilon_{\text{approximation}}/(T \cdot d)$.
- D_{arith} is the T-depth of the arithmetic module that accumulates the affine-transformed log-return as explained in the re-parameterization method and the different relevant comparators of the algorithm, as in [5], as well as the constant payoffs calculation (for binary options and the zero payoff).
- D_{exp} is the T-depth of the partial exponential state preparation for the integration method, which happens in parallel to the Gaussian preparation and arithmetic.
- D_{AL} is the amplitude loading T-depth.

A. Rescaling Analysis

Rescaling is one of the critical parameters that affects the T-depth. A given payoff error, ϵ_{payoff}, corresponds to the following error ϵ in the probability domain:

$$\epsilon = \frac{\epsilon_{\text{payoff}}}{R} \tag{25}$$

where R is the normalization factor applied in the post-processing stage, that amplifies ϵ accordingly.

The rescaling factor for the integration method is:

$$R = f_{\max} + (K - r_{T_{\min}})V \tag{26}$$

where $f_{\max}$ is the maximal binary payoff; see Section III-C.

B. Complexity and Error Analysis of Gaussian Preparation

We now analyze the size of the Gaussian preparation blocks for each time step based on the payoff error. The sources of error in this block are the approximation error, truncation error, and discretization error.

The Gaussian preparation block must be chosen such that these sources of error do not exceed ϵ. This requires choosing a suitable distribution truncation and an appropriate number of qubits per block, k, to capture a multidimensional Gaussian distribution for T time steps and d assets. The distribution has $\sigma_{\max}$ as the maximal eigenvalue of the covariance matrix for a unit time. Following [5] we define w, the number of $\sigma_{\max}$ we use for the truncation to ensure $\epsilon_{\text{truncation}} \leq \epsilon$. We require that $P(r_T < r_{T_{\min}}) \leq \epsilon_{\text{truncation}}/2$; this condition determines the minimum return at time T as $r_{T_{\min}} = \exp(\mu \Delta t T - w\sigma_{\max}\sqrt{\Delta t T})$, where Δt is the unit time per sample point. Once we have w, we can use it to find k, ensuring $\epsilon_{\text{discretization}} \leq \epsilon$.

Algorithm 1 Quantum algorithm for pricing autocallable options

Input: Mean of historical log-return μ. Standard deviation of historical log-return σ. Gaussian discretization step ds, number of discretization qubits n and truncation value s_{min}. Time step size dt. Total number of time steps T. Barrier b and strike return K for the put option. Amplitude-mapped payoff $\tilde{f}_i$ for binary i with strike k_i. Amplitude-mapped zero payoff $\tilde{f}_z$.

Load a Gaussian distribution for each time step:

$$|G\rangle_{Tn} = \bigotimes_{i=1}^{T} G|0\rangle_n = \bigotimes_{i=1}^{T} \left(\sum_{g=0}^{2^n-1} \sqrt{p(g_i)}|g_i\rangle_n \right) \tag{16}$$

where g_i is the g-th discretization point of the i discrete standard Gaussian and $p(g_i)$ is the respective probability.

Prepare an accumulator register of size m (sufficient to store cumulative log-returns up to time T) and compute the log-return for the first time step (l_1) using the operator R_1:

$$|G\rangle_{Tn}|0\rangle_m \xrightarrow{R_1} |G\rangle_{Tn}|l_1\rangle_m = |G\rangle_{Tn}|l(g_1)\rangle_m \tag{17}$$

where the function $l(g)$ adjusts the mean and variance of the Gaussian sample:

$$l(g) = \mu \cdot dt + \sigma(g \cdot ds + s_{min}) \cdot \sqrt{dt} \tag{18}$$

for t in $[1, \ldots, T]$ **do**

  if $t \neq 1$ **then**

    Compute the log-return l_t at time t through an operator R_t by applying the function in Equation 18 to g_t, and then adding the log-return from the previous time step (l_{t-1}):

$$|G\rangle_{Tn}|l_{t-1}\rangle_m \xrightarrow{R_t} |G\rangle_{Tn}|l_t\rangle_m = |G\rangle_{Tn}|l_{t-1} + l(g_t)\rangle_m \tag{19}$$

  end if

  Update the register tracking the barrier crossing event $|C_t\rangle_T = |c_1 c_2 ... c_T\rangle$. The initial value for the register is $|C_0\rangle_T = |0\rangle_T$. The operator C_t sets the t-th qubit to $|1\rangle$ if the barrier is crossed at time t (i.e., $c_t = (l_t < \ln(b))$):

$$|G\rangle_{Tn}|l_t\rangle_m|C_{t-1}\rangle_T \xrightarrow{C_t} |G\rangle_{Tn}|l_t\rangle_m|C_t\rangle_T \tag{20}$$

  Update the register tracking the verification of binaries' conditions $|B_t\rangle_j = |b_0 b_1 ... b_{j-1}\rangle$ initialized as $|B_0\rangle_j = |0\rangle_j$. Let $\{s_0, s_1 ... s_{j-1}\} \subset [1, T-1]$ be the times at which binaries expire. b_i is set to 1 if the binary defined at time $t = s_i$ is in the money and if all of the previous ones were out of the money (i.e., $b_i = (l_{s_i} > \ln(k_i)) \wedge (b_h = 0 \quad \forall h < i)$). Let B_j be the operator that performs this update:

$$|G\rangle_{Tn}|l_t\rangle_m|C_t\rangle_T|B_{t-1}\rangle_j \xrightarrow{B_j} |G\rangle_{Tn}|l_t\rangle_m|C_t\rangle_T|B_t\rangle_j \tag{21}$$

  Apply a controlled R_y rotation via b_i using operator F_i to update the amplitude of the target qubit (with initial state $|\tilde{f}_0\rangle = |0\rangle$) with the binary payoff (if $b_i = 1$):

$$|G\rangle_{Tn}|l_t\rangle_m|C_T\rangle_T|B_t\rangle_j|\tilde{f}_{t-1}\rangle \xrightarrow{F_i} |G\rangle_{Tn}|l_t\rangle_m|C_T\rangle_T|B_t\rangle_j|\tilde{f}_t\rangle = |G\rangle_{Tn}|l_t\rangle_m|B_t\rangle_j(\sqrt{\tilde{f}_i}\,|1\rangle + \sqrt{1-\tilde{f}_i}\,|0\rangle) \tag{22}$$

end for

Apply a controlled operator F under the following conditions: no binary option was in the money ($|B_T\rangle_j = |0\rangle_j$), and the barrier was crossed at least once ($|C_T\rangle_T \neq |0\rangle_T$), and the put is in the money ($l_T < ln(K)$). The operator performs an exponential amplitude loading (defined in Section III-B) to encode the amplitude-mapped non-zero payoff for the put option ($\tilde{f}$) in the target qubit:

$$|G_T\rangle|l_t\rangle_m|C_T\rangle_T|B_T\rangle_j|\tilde{f}_T\rangle \xrightarrow{F} |G\rangle_{Tn}|l_t\rangle_m|C_T\rangle_T|B_T\rangle_j(\sqrt{\tilde{f}(l_T)}\,|1\rangle + \sqrt{1-\tilde{f}(l_T)}\,|0\rangle) \tag{23}$$

Apply a controlled R_y rotation using the operator F_z to encode $\tilde{f}_z$ (the value needed to obtain a 0 payoff after postprocessing) when none of the above conditions are met:

$$|G\rangle_{Tn}|l_t\rangle_m|C_T\rangle_T|B_T\rangle_j|\tilde{f}_T\rangle \xrightarrow{F_z} |G\rangle_{Tn}|l_t\rangle_m|C_T\rangle_T|B_T\rangle_j(\sqrt{\tilde{f}_z}\,|1\rangle + \sqrt{1-\tilde{f}_z}\,|0\rangle) \tag{24}$$

	Function	T-depth	Description
D_{R_y}	R_y with error ϵ	$3\log_2 \frac{1}{\epsilon}$	According to [20]
D_{CR_y}	CR_y with error ϵ	$6\log_2 \frac{2}{\epsilon}$	A CR_y can be implemented by 2 consequent applications $R_y(\theta/2)$ and 2 CX
D_{R_z}	R_z with error ϵ	$\log_2 \frac{1}{\epsilon}$	According to [20]
D_{Toffoli}	Toffoli	3	According to [21]
$D_{\text{MCX}}(n)$	Muti Controlled X (MCX) with n control qubits	$14\log_3 \frac{n}{2} + 5$	The T-depth of the MCX is reduced using relative Toffoli gates [22], and further optimized by the Classiq platform [23]
$D_{\text{comparator}}(n)$	Comparator of n qubits	$(2\log_2(n) + 9)D_{\text{Toffoli}}$	According to [21]
$D_{\text{C-comp}}(n)$	Controlled Comparator of n qubits	$D_{\text{comparator}}(n) + D_{\text{MCX}}(3) - D_{\text{Toffoli}}$	A controlled comparator can be implemented by controlling just the middle Toffoli gate of the comparator (resulting in a CCCX). The T-depth can then be derived from the one of the comparator
$D_{\text{adder}}(n)$	Adder of n qubits	$(2\log_2(n) + 5)D_{\text{Toffoli}}$	According to [21]
$D_{\text{multiplier}}(n)$	Multiplier of n qubits	$n(D_{\text{adder}}(n) + 6)$	According to [24]

TABLE I: T-Depth of Basic Building Blocks.

1) Finding the Required Truncation : The challenge in finding w relies on the fact that ϵ depends on R according to Equation 25. Additionally, R depends on $r_{T_{\min}}$ (see Equation 26), which, in turn, depends on w.

The truncation error is the sum of all $2dT$ tails of the truncated Gaussian distributions [5]:

$$\epsilon_{\text{truncation}} = 2dTe^{-w^2/2} \tag{27}$$

One must find w that satisfies:

$$\epsilon_{\text{truncation}} \leq \epsilon = \frac{\epsilon_{\text{payoff}}}{f_{\max} + (K - r_{T_{\min}})V} \tag{28}$$

2) Gaussian Preparation Depth Analysis: Once we find the truncation parameter w, for each Gaussian preparation block we can determine the number of qubits k that discretizes the truncated domain while maintaining the discretization error $\epsilon_{\text{discretization}}$; following the approach described in [5].

The Gaussian preparation circuit on k qubits is a variational circuit with L layers. Each layer contains an R_y rotation on each qubit. Therefore, the T-depth of each R_y, D_{R_y}, depends on the desired approximation error by $\epsilon_{R_y} = \epsilon_{\text{approximation}}/(k \cdot T \cdot d)$. Thus, the total T-depth will be [5]:

$$D_{\text{G}} = (L + 1) \cdot D_{R_y}. \tag{29}$$

C. Arithmetic and Constant Payoff Modules Complexity

Following the Gaussian preparation, we compute the log-return evolution over time, verify payoff conditions, and compute the constant payoffs. The log-return evolution and condition evaluation are performed using arithmetic operations, while a combination of MCX and CR_y gates is used to store constant payoffs in the amplitude of the target qubit according to the relevant conditions. The T-depth of these operations (D_{arith}) can be constructed from the elementary building blocks in Table I.

D. Complexity of Amplitude Loading Module

Our integration amplitude loading module (see Section III-B) is controlled to identify the condition for non-zero put payoff activation. The module consists of a partial exponential state preparation block followed by an integration comparator (see Figure 1a). The entire module can be controlled by applying control only to the integration comparator, as the partial exponential state preparation operates solely on the r-register. Since the partial exponential state preparation can be executed in parallel with the rest of the circuit, the amplitude loading complexity is determined solely by that of the controlled comparator:

$$D_{\text{AL}} = D_{\text{C-comp}}(m) \tag{30}$$

where m is the number of qubits required to store the log-return at the final time. The complexity of the partial exponential state preparation, $D_{\exp}$, is derived as follows.

1) Complexity of partial exponential state preparation : The partial exponential state preparation module (depicted in Figure 2b) runs in parallel to the modules described in Sections IV-B and IV-C. Its depth depends on m and on the ϵ requirement:

$$D_{\exp} = 3D_{Ry} + D_{\text{MCX}}(m) + 2D_{\text{C-comp}}(m) \tag{31}$$

The 3 exponential state preparation blocks contain an R_y rotation on each of the m qubits as well as on the additional auxiliary qubit, totaling at $m+1$ qubits. Therefore, the T-depth, D_{R_y}, of each R_y depends on the error by $\epsilon_{\text{amplitude loading}}/(m+1)$. The diffuser block contains an MCX, and there are two controlled comparators, one for each inequality; their depths are given in Table I.

E. Comparison with State of the Art

We provide a T-depth analysis of our proposed autocallable pricing algorithm and compare it with an implementation based on QSP [6]. For a fair comparison, we consider the modules presented in Sections IV-B and IV-C for both approaches, changing only the amplitude loading module, which constitutes the primary distinguishing feature between the two methods. The problem parameters are chosen to be the same

as in [6], with $T = 20$ timesteps, $d = 3$ assets and $\epsilon = 2 \cdot 10^{-3}$ total estimation error.

For the integration amplitude loading method proposed in [7], normalization diminishes any improvement obtained from amplitude loading. Instead, the method presented in Section III-B shows a significant improvement for the amplitude loading module, from $\sim 2.1 \cdot 10^3$ to ~ 40 T-depth (considering the parallelization of the partial exponential state preparation module), without significantly altering the depth of the other modules. Such a significant enhancement in the T-depth necessary for the payoff component suggests that further improvements to the overall T-depth are unlikely to come from optimizing the amplitude loading module alone. However, it opens the door to further improvements in other components of the quantum algorithm.

V. Experiments

This section presents the results of an extensive series of simulation experiments aimed at validating the proposed model. The input data for replicating such experiments can be found in Table II. The implementation is intended to be a proof of concept for the integration methodology in the context of autocallable option pricing. The full algorithm for pricing autocallable options is based on the method described in III-A to avoid implementing different problem-dependent models required by the method reported in III-B. However, a code base implementing the different strategies for loading the exponential state preparation is also available together with the whole algorithm [25] and can be used to adapt the implementation to obtain optimal rescaling, given the problem parameters.

We used the Classiq platform to build a high-level functional model and synthesize it to obtain the resulting gate-based quantum circuit [8], [26]. The Classiq synthesis engine explores multiple implementations with the same functionality and returns an optimized circuit based on user-defined preferences, such as optimizing circuit depth, number of qubits, or two-qubit gate usage, or tailoring the circuit for specific hardware [27].

Currently available quantum computers have limited coherence times and their fidelity is not yet sufficient for large-scale circuits executions. This makes them unsuitable for validating the proposed algorithm. For this reason, we employed quantum simulators to validate our approach. To enable circuit execution on simulators, we synthesized the circuit minimizing its width to reduce the number of qubits required.

For our experiments, we considered an autocallable option with a single asset, three time steps, and two binary options expiring at the first two monitoring points.

The goal of the quantum algorithm is to improve the convergence rate compared to traditional Monte Carlo methods. Therefore, the ideal validation for the quantum circuit should consist in comparing its execution outcomes with those obtained using this classical methodology. However, a fair comparison is not feasible on current simulators because quantum circuits synthesized under the accuracy constraints outlined in Section IV exceed the simulators' capabilities. In fact, existing simulators can only handle quantum circuits with very limited width and depth.

TABLE II: Model Parameters for Reproducibility

Parameter	**Value**
Notional value (V)	18$
Time step (dt)	1 year
Number of time steps (T)	3
Annual volatility (σ)	0.2382
Annual average log-return (μ)	0.1274
Put barrier (b)	0.7
Put strike return (K)	1
Risk-free rate (r)	4%
First binary option strike (k_1)	1.1
Second binary option strike (k_2)	1.1
First binary option payoff (f_1)	2
Second binary option payoff (f_2)	5
Standard Gaussian truncation (s_{min})	3

To address this limitation, we built a set of classical models to experimentally demonstrate that it is possible to achieve convergence to the traditional Monte Carlo result by starting with a circuit executable on current simulators and then upgrading it, increasing the number of qubits allocated for Gaussian encoding and quantum arithmetic precision.

We conducted comparisons using (1) the traditional Monte Carlo methodology, (2) a Monte Carlo methodology relying on the same Gaussian discretization of the quantum algorithm, (3) a closed form calculation constrained by the same Gaussian discretization and (4) a closed form considering both the Gaussian discretization and the arithmetic precision of the quantum model.

The fourth benchmarking approach was essential to validate our quantum circuit results by comparing them with a classical model subject to the same constraints. In fact, we observed that the results of the classical algorithm fell within the confidence interval provided by the iterative version of the QAE [16]. We set the target confidence level choosing $\epsilon = 0.001$ and $\alpha = 0.002$. The crosses in Figure 3 represent the results of executions performed on exact shot-based simulators. For up to 25 qubits, we used the default Classiq simulator. For up to 30 qubits, we used the Nvidia simulator on a single GPU. For circuits exceeding 30 qubits, we executed the program on the LEONARDO HPC system [9], using the Cirq simulator on top of a cuQuantum appliance with multiple GPUs.

The third classical algorithm shows the behavior that the quantum circuit would exhibit if the number of precision qubits for digital encoding matched the classical machine precision. As shown in Figure 3, increasing the number of precision qubits causes the fourth classical model (benchmarked with the quantum algorithm) to converge with the third. The third classical closed form algorithm, which still incorporates Gaussian discretization, becomes impractical as the number

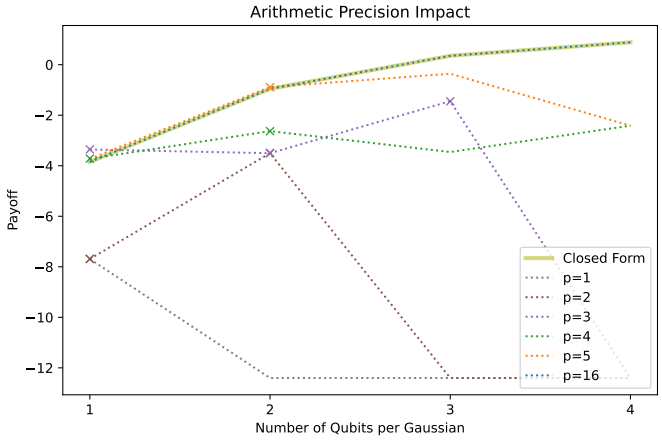


Fig. 3: Expected payoff for the following classical models: (1) closed form with Gaussian discretization, (2) closed form with Gaussian discretization and explicit fixed point precision (where p denotes the number of qubits used for the fractional part). The crosses 'x' represent results of the execution on simulators of the quantum circuit, synthesized with the given precision and discretization qubits. The configurations $[p = 2, n_{\text{gaussian qubits}} = 1]$, $[p = 3, n_{\text{gaussian qubits}} = 1]$ were executed on Classiq default simulator; $[p = 4, n_{\text{gaussian qubits}} = 1]$, $[p = 3, n_{\text{gaussian qubits}} = 2]$, $[p = 4, n_{\text{gaussian qubits}} = 2]$ were executed on the Nvidia simulator; while $[p = 5, n_{\text{gaussian qubits}} = 2]$ and $[p = 3, n_{\text{gaussian qubits}} = 3]$ were executed on LEONARDO's cluster.

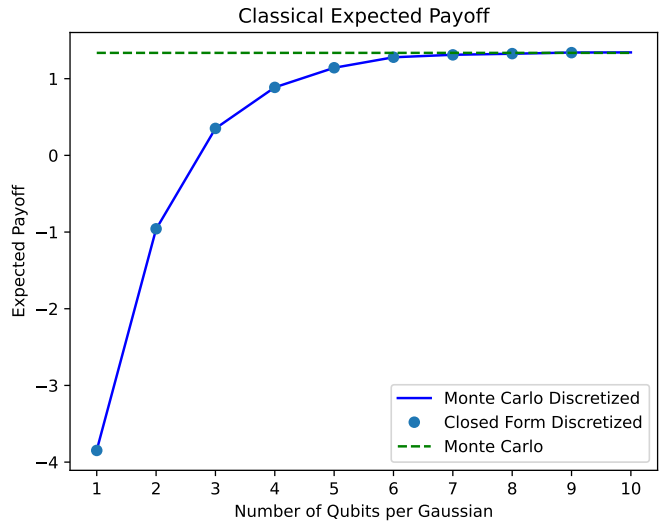


Fig. 4: Expected payoff under the following methods: (1) traditional Monte Carlo simulation, (2) Monte Carlo simulation accounting for the number of discretization qubits for the Gaussian distribution, and (3) closed form calculation with the same discretization.

of discretization points and time instances increases, requiring prohibitive computational and memory resources. However, we can verify from Figure 4 that the results of a Monte Carlo algorithm considering the same Gaussian discretization overlap with the ones obtained with the closed form calculation. In the same figure, a verification of convergence of the two Monte Carlo methodologies (the discretized and non-discretized one) can also be observed. Through these tests, we had the opportunity to demonstrate the convergence of our results to those of the traditional Monte Carlo methodology, once it becomes possible to increase the number of qubits underlying the quantum circuits.

VI. Conclusion

We presented a novel approach for pricing autocallable options. This includes a full end-to-end implementation using the Classiq platform and the Qmod language. Our methodology builds upon the integration-based exponential amplitude loading introduced in [7] and enhances it by not degrading the normalization factor.

Furthermore, we present a comprehensive complexity analysis to compare our approach with the state-of-the-art in the literature, particularly with the implementation based on Quantum Signal Processing in [6]. From this comparison, we saw a significant decrease in the necessary depth in terms of T-gates for the subroutine of amplitude loading. Specifically, using our method we obtain a ~ 50x reduction in a relevant setting.

We also presented the results of extensive simulation campaigns using, among others, the Nvidia simulator and the LEONARDO supercomputer. These results validate our approach and illustrate how, by increasing the qubits allocated to arithmetic precision and Gaussian discretization, the behavior fits expected outcomes more effectively, enabling progressively closer values to those obtained with the classical Monte Carlo methodology.

The main challenge remains the considerable size of the circuits obtained even for small-scale examples, which necessitates the use of simulators, making potential tests on real hardware less interesting.

In the future, further optimization aimed at reducing the circuit size is desirable, as well as initiating a campaign of tests on real hardware that leverages error mitigation techniques to preserve the output. Additionally, the extension of the methodology to more complex volatility models could be explored.

Acknowledgment

We acknowledge the ICSC for awarding this project access to the EuroHPC supercomputer LEONARDO, hosted by CINECA (Italy). The authors thank Chani Sacharen for the valuable help in editing and proofreading the paper, ensuring clarity and coherence throughout the manuscript.

References

[1] R. Orús, S. Mugel, and E. Lizaso, "Quantum computing for finance: Overview and prospects," *Reviews in Physics*, vol. 4, p. 100028, Nov. 2019. [Online]. Available: http://dx.doi.org/10.1016/j.revip.2019.100028

[2] D. J. Egger, C. Gambella, J. Marecek, S. McFaddin, M. Mevissen, R. Raymond, A. Simonetto, S. Woerner, and E. Yndurain, "Quantum computing for finance: State-of-the-art and future prospects," *IEEE Transactions on Quantum Engineering*, vol. 1, p. 1–24, 2020. [Online]. Available: http://dx.doi.org/10.1109/TQE.2020.3030314

[3] D. Herman, C. Googin, X. Liu, Y. Sun, A. Galda, I. Safro, M. Pistoia, and Y. Alexeev, "Quantum computing for finance," *Nature Reviews Physics*, vol. 5, no. 8, p. 450–465, Jul. 2023. [Online]. Available: http://dx.doi.org/10.1038/s42254-023-00603-1

[4] R. Sotelo, D. Corbelletto, E. Dri, E. Giusto, and B. Montrucchio, "Quantum computing in finance: The intesa sanpaolo experience," *IEEE Engineering Management Review*, vol. 52, no. 2, p. 9–15, Apr. 2024. [Online]. Available: http://dx.doi.org/10.1109/EMR.2024.3373796

[5] S. Chakrabarti, R. Krishnakumar, G. Mazzola, N. Stamatopoulos, S. Woerner, and W. J. Zeng, "A threshold for quantum advantage in derivative pricing," *Quantum*, vol. 5, p. 463, Jun. 2021. [Online]. Available: http://dx.doi.org/10.22331/q-2021-06-01-463

[6] N. Stamatopoulos and W. J. Zeng, "Derivative Pricing using Quantum Signal Processing," *Quantum*, vol. 8, p. 1322, Apr. 2024. [Online]. Available: https://doi.org/10.22331/q-2024-04-30-1322

[7] F. Cibrario, O. S. Golan, G. Ranieri, E. Dri, M. Ippoliti, R. Cohen, C. Mattia, B. Montrucchio, A. Naveh, and D. Corbelletto, "Quantum amplitude loading for rainbow options pricing," in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Sep. 2024, p. 211–220. [Online]. Available: http://dx.doi.org/10.1109/QCE60285.2024.00034

[8] "Qmod - the quantum modeling language," https://docs.classiq.io/latest/qmod-reference/, 2024.

[9] M. Turisini, M. Cestari, and G. Amati, "Leonardo: A pan-european pre-exascale supercomputer for hpc and ai applications," *Journal of large-scale research facilities JLSRF*, vol. 9, no. 1, Jan. 2024. [Online]. Available: http://dx.doi.org/10.17815/jlsrf-8-186

[10] J. Hull, *Options, futures, and other derivatives*, 7th ed., ser. The Prentice Hall series in finance. Upper Saddle River, NJ: Pearson Prentice Hall, 2009.

[11] A. Montanaro, "Quantum speedup of monte carlo methods," *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 471, no. 2181, p. 20150301, Sep. 2015. [Online]. Available: http://dx.doi.org/10.1098/rspa.2015.0301

[12] P. Rebentrost, B. Gupt, and T. R. Bromley, "Quantum computational finance: Monte carlo pricing of financial derivatives," *Physical Review A*, vol. 98, no. 2, Aug. 2018. [Online]. Available: http://dx.doi.org/10.1103/PhysRevA.98.022321

[13] N. Stamatopoulos, D. J. Egger, Y. Sun, C. Zoufal, R. Iten, N. Shen, and S. Woerner, "Option pricing using quantum computers," *Quantum*, vol. 4, p. 291, Jul. 2020. [Online]. Available: http://dx.doi.org/10.22331/q-2020-07-06-291

[14] K. Kaneko, K. Miyamoto, N. Takeda, and K. Yoshino, "Quantum pricing with a smile: implementation of local volatility model on quantum computer," *EPJ Quantum Technology*, vol. 9, no. 1, p. 7, Dec. 2022. [Online]. Available: https://epjquantumtechnology.springeropen.com/articles/10.1140/epjqt/s40507-022-00125-2

[15] G. Wang and A. Kan, "Option pricing under stochastic volatility on a quantum computer," *Quantum*, vol. 8, p. 1504, Oct. 2024. [Online]. Available: https://doi.org/10.22331/q-2024-10-23-1504

[16] D. Grinko, J. Gacon, C. Zoufal, and S. Woerner, "Iterative quantum amplitude estimation," *npj Quantum Information*, vol. 7, no. 1, Mar. 2021. [Online]. Available: http://dx.doi.org/10.1038/s41534-021-00379-1

[17] G. L. Long, "Grover algorithm with zero theoretical failure rate," *Physical Review A*, vol. 64, no. 2, Jul. 2001. [Online]. Available: http://dx.doi.org/10.1103/PhysRevA.64.022307

[18] S. McArdle, A. Gilyén, and M. Berta, "Quantum state preparation without coherent arithmetic," 2022. [Online]. Available: https://arxiv.org/abs/2210.14892

[19] P. Niemann, A. Gupta, and R. Drechsler, "T-depth optimization for fault-tolerant quantum circuits," in *2019 IEEE 49th International Symposium on Multiple-Valued Logic (ISMVL)*. IEEE, 2019, pp. 108–113.

[20] N. J. Ross and P. Selinger, "Optimal ancilla-free clifford+ t approximation of z-rotations," *arXiv preprint arXiv:1403.2975*, 2014.

[21] T. G. Draper, S. A. Kutin, E. M. Rains, and K. M. Svore, "A logarithmic-depth quantum carry-lookahead adder," *arXiv preprint quant-ph/0406142*, 2004.

[22] D. Maslov, "Advantages of using relative-phase toffoli gates with an application to multiple control toffoli optimization," *Physical Review A*, vol. 93, no. 2, p. 022311, 2016.

[23] "Classiq: High-level quantum modeling language," https://github.com/Classiq/classiq-library, 2024.

[24] K. Karuppasamy, V. Puram, S. Johnson, and J. P. Thomas, "A comprehensive review of quantum circuit optimization: Current trends and future directions," *Quantum Reports*, vol. 7, no. 1, 2025. [Online]. Available: https://www.mdpi.com/2624-960X/7/1/2

[25] "Quantum autocallable implementation," http://short.classiq.io/quantum_autocallable.

[26] M. Vax, P. Emanuel, E. Cornfeld, I. Reichental, O. Opher, O. Roth, T. Michaeli, L. Preminger, L. Gazit, A. Naveh *et al.*, "Qmod: Expressive high-level quantum modeling," *arXiv preprint arXiv:2502.19368*, 2025.

[27] T. Goldfriend, I. Reichental, A. Naveh, L. Gazit, N. Yoran, R. Alon, S. Ur, S. Lahav, E. Cornfeld, A. Elazari, P. Emanuel, D. Harpaz, T. Michaeli, N. Erez, L. Preminger, R. Shapira, E. M. Garcell, O. Samimi, S. Kisch, G. Hallel, G. Kishony, V. v. Wingerden, N. A. Rosenbloom, O. Opher, M. Vax, A. Smoler, T. Danzig, E. Schirman, G. Sella, R. Cohen, R. Garfunkel, T. Cohn, H. Rosemarin, R. Hass, K. Jankiewicz, K. Gharra, O. Roth, B. Azar, S. Asban, N. Linkov, D. Segman, O. Sahar, N. Davidson, N. Minerbi, and Y. Naveh, "Design and synthesis of scalable quantum programs," 2024.